Algorithms 1
Homework 5

Your Name

13 октября 2024 г.

1 Статистики, битовые операции, нижние оценки

1. Для решения данной задачи можно хранить две переменные: cur_elem, $running_count$, которые будут хранить текущий элемент и количество его вхождений. Алгоритм итерируется по массиву и на каждом шаге проверяет, равен ли текущий элемент cur_elem. Если равен, то увеличиваем $running_count$, иначе уменьшаем $running_count$. Если $running_count = 0$, то присваиваем cur_elem текущему элементу. В конце алгоритма искомый элемент будет храниться в cur_elem. Этот алгоритм работает за $O(n)$ времени и $O(1)$ памяти.

```python
def majority_element(arr):
    cur_elem = None
    running_count = 0
    for elem in arr:
        if running_count == 0:
            cur_elem = elem

        running_count += 1 if cur_elem == elem else -1

    return cur_elem
```

2. Асимптотика алгоритма поиска k-той порядковой статистики при разбиении массива на подмассивы из p элементов:

$$T(n) = T(\frac{n}{p}) + Cn + T(\frac{3p-1}{4p}n)$$

где C — константа, n — размер массива, Если предположить, что сортировка p элементов занимает $O(1)$ времени. Асимптотика будет представлять собой дерево рекурсии глубины от $\log_p n$ до $\log_{\frac{4p}{3p-1}} n$, асимптотика алгоритма на i-ом уровне рекурсии (считая с вершины дерева) будет равна $Cn(\frac{3p+3}{4p})^i$, таким образом Асимптотика алгоритма будет равна

$$T(n) = O(\sum_{i=1}^{\lceil \log_{\frac{4p}{3p-1}} n \rceil} n(\frac{3p+3}{4p})^i)$$

Для $\frac{3p+3}{4p} < 1$ сумма будет ограничена сверху константой, тогда асимптотика будет равна $O(n)$.

Подставим $p = 3$:

$$T(n) = O(\sum_{i=1}^{\lceil \log_{\frac{3}{2}} n \rceil} n1^i) = O(n \log n)$$

Подставим $p = 7$:

$$T(n) = O(\sum_{i=1}^{\lceil \log_{\frac{7}{5}} n \rceil} n(\frac{6}{7})^i) = O(n)$$

3. Этот процесс эквивалетен поиску наибольшего общего делителя алгоритмом евклида. Рассмотрим пару чисел a, b, без ограничения общности $a > b$. Тогда $a = bq + r$, после применения данной операции q раз пара чисел станет равна $b, a \mod b$. Теперь докажем, что этот процесс сводится к применению алгоритма евклида попарно ко всем элементам. Заметим, что $\gcd(a, b) = \gcd(a - b, b)$, тогда процесс можно представить как применение этого процесса сначала к a_1, a_2 до нахождения их gcd, затем к a_1, a_3 и так далее. Таким образом, процессс-водится к поиску gcd попарно ко всем э лементам, и он гарантированно сойдется к gcd всех элементов массива.

4. Пусть x и y - битовые длины данных чисел. Тогда асимптотическая сложность этого алгоритма:
$$T(x, y) = T(x - 1, y) + O(x + y)$$
Так как рекурсивный вызов происходит с уменьшением длины одного из чисел на 1, после чего с обоими числами проводятся линейные операции: сложение, вычитание, сравнение, умножение на 2 чисел q и r, линейно зависящих от x. Тогда асимптотика данного алгоритма:
$$T(x, y) = O(x(x + y))$$

5. Для примера будем использовать алгоритм быстрого возведения в степень:

```python
def power(a, n, m): # n is bit representation
    res = 1
    a = a % m
    for i in range(len(n)):
        if n[i] == "1":
            res = (res * a) % m
        a = (a * a) % m

    return res
```

Возьмём a, n, m - длины битовых представлений чисел a, n, m. Тогда асимптотика данного алгоритма:
$$T(a, n, m) == O(na(a + m))$$
Алгоритм проходит по всем битам числа n, на каждой итерации в худшем случае производит умножение числа a на res по модулю m, и умножение на a по модулю m, умножение происходит за $O(a^2)$ деление по модулю за $O(a(a + m))$.

6. Оптимальный алгоритм делит исходную кучу пополам, проверяет каждую счётчиком за константу, потом рекурсивно проводит ту же операцию для кучи, где счётчик показал радиацию. Такой алгоритм будет работать за
$$O(\lceil \log n \rceil)$$
Если считать время измерения счётчиком за константу. Докажем, что меньшее количество операций сделать невозможно. Пусть одно применение счётчика присваивает значение 1 ко всем камням кучи, где найдена радиация, и 0 камням, где радиации нет. Тогда по завершению процесса каждому камню будет присвоена уникальная битовая маска длины $\lceil \log n \rceil$. Предположим, что можно было бы провести меньше операций, например, $\lceil \log n \rceil - 1$. Тогда по принципу Дирихле найдутся два камня с одинаковыми масками, что противоречит тому, что каждому камню присвоена уникальная маска. Таким образом, оптимальное количество операций равно $\lceil \log n \rceil$.